基于MPI的基本矩阵运算以及优化方案



2025

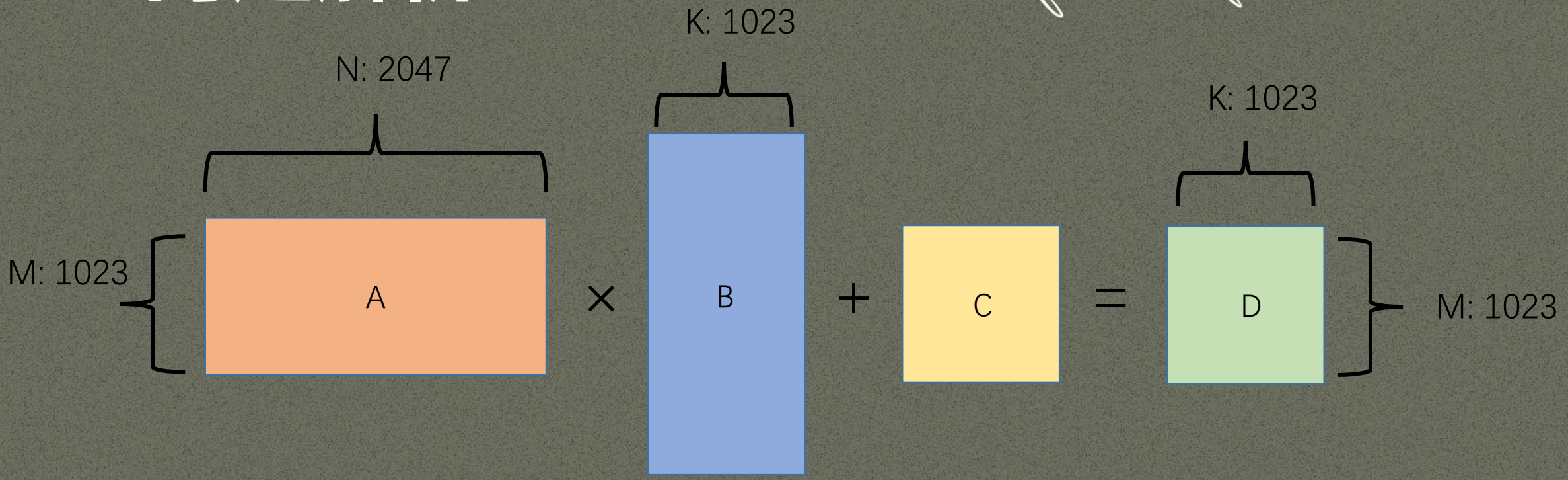
问题回顾

- 任务：大规模矩阵运算
- D = A * B + C
  A: 1023x2047, B: 2047x1023, C: 1023x1023
- 采用MPI（Message Passing Interface）消息传递接口，并行计算
- 对比 2 核与 4 核的并行加速比与效率

2025

问题解析

$D = A \times B + C$

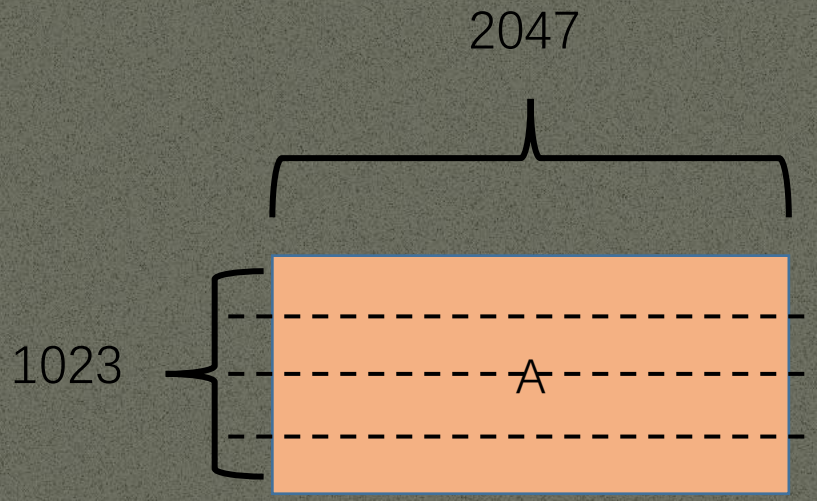


计算D的每个元素，由A的每行×B的每列+对应位置的C得到

计算量太大

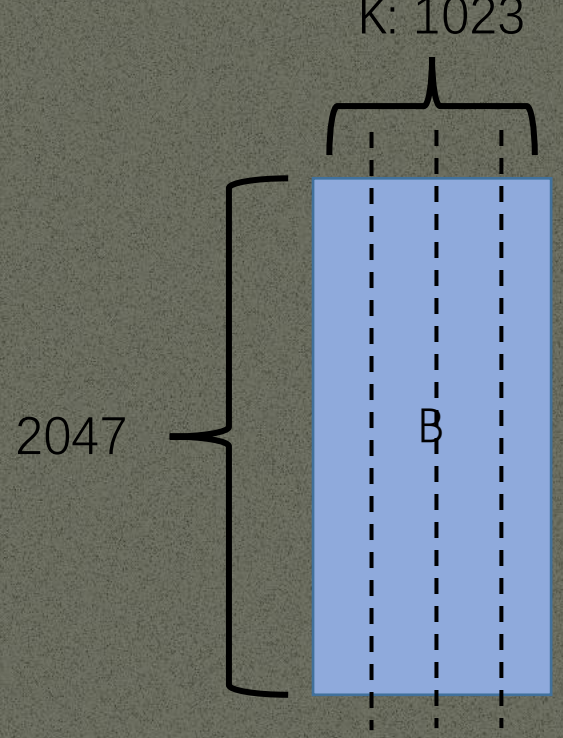
如何划分 $D = A \times B + C$

按行划分



计算D的一个行，只需用到A的一行，C的一行，和整个B

按列划分

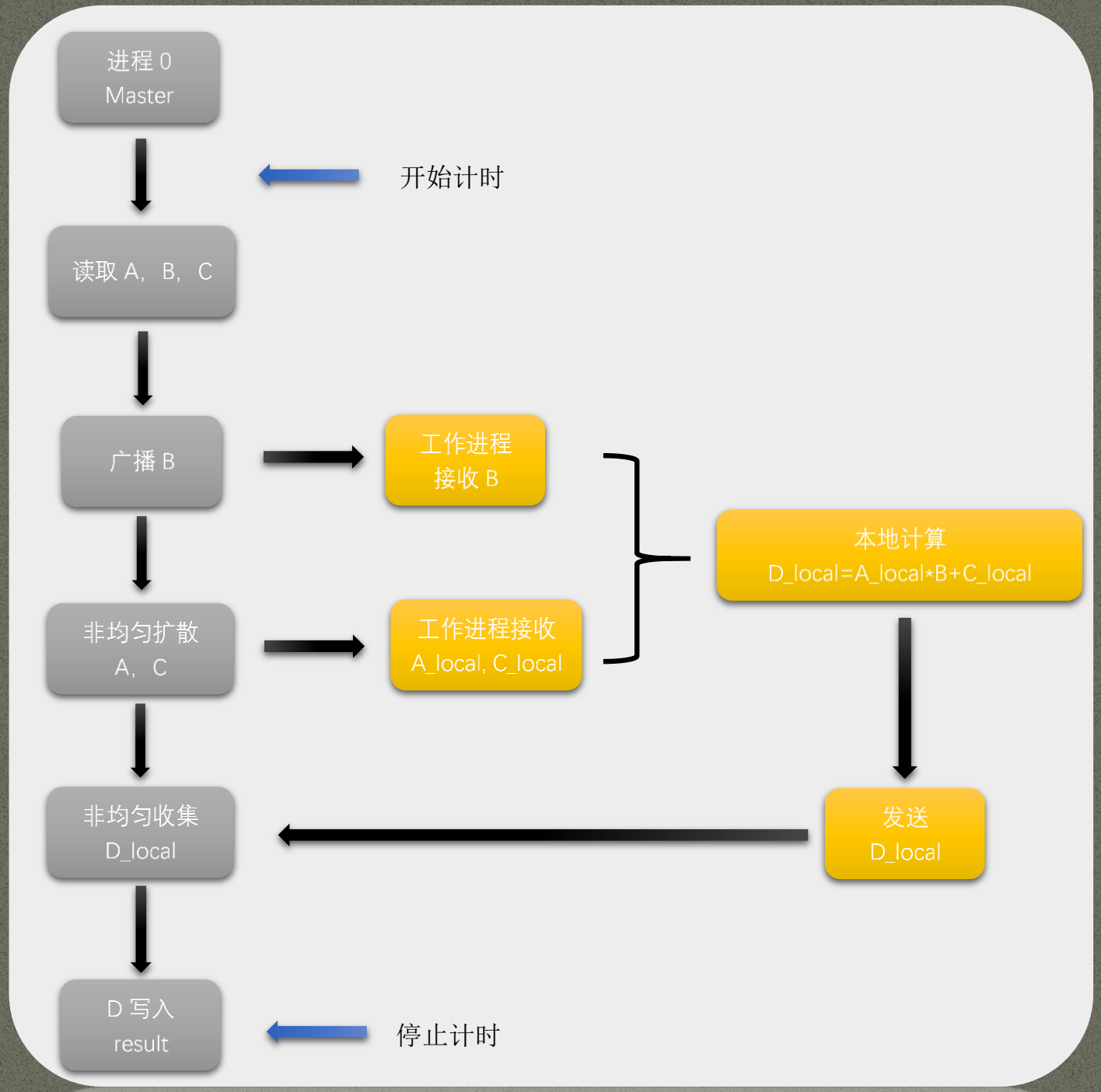


计算D的一个列，只需用到B的一列，C的一行，和整个A

由于数组在计算机内存中是行主序存储的，也就意味着矩阵的一行，在内存中是连续的一片区域，一列则是跳跃分布的

MPI方案

- 采用主-从模式，按行分解任务
- 数据分发：广播矩阵B，分配给所有进程；矩阵A，C，非均匀散播，确保**负载均衡**
- 本地并行计算：D_local = A_local * B + C_local
- 结果收集，非均匀收集，为非均匀散播的逆过程，拼成完整D



关键代码分析

```
int main(int argc, char* argv[]) {
    double start_time, end_time;
    start_time = MPI_Wtime();

    MPI_Init(&argc, &argv);// 启动MPI环境

    int world_rank, world_size;
    // 获取当前进程编号（0-3）
    MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);
    // 获取总进程数（4）
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);

    Matrix A, B, C, D;

    if (world_rank == 0) {

        // 进程 0 读取所有数据
        readMatrices("RandMatrix.txt", A, B, C);
        D.resize(M * K);
    } else {
        // 其他进程分配 B 的空间
        B.resize(N * K);
    }
```

开始计时

获取进程编号和数量
通过编号识别身份

如果当前是0号进程，则
读取全部数据，分配空间

关键代码分析

由进程0不均匀分发A，C

每个进程最少行：255
不被整除剩余行：3

发送数量/进程需要
元素个数

```
int base_rows = M / world_size;
int remainder = M % world_size;
int current_row = 0;

for (int i = 0; i < world_size; ++i) {
    int rows_for_this_proc = base_rows + (i < remainder ? 1 : 0);

    sendcounts_A[i] = rows_for_this_proc * N; // A 的元素个数
    displs_A[i] = current_row * N;           // A 的偏移量

    sendcounts_C[i] = rows_for_this_proc * K; // C 和 D 的元素个数
    displs_C[i] = current_row * K;           // C 和 D 的偏移量

    current_row += rows_for_this_proc;
}
```

```
// 使用 Scatterv 分发 C
MPI_Scatterv(
    world_rank == 0 ? C.data() : nullptr, // 仅 rank 0 发送
    sendcounts_C.data(),
    displs_C.data(),
    MPI_LONG_LONG,
    C_local.data(),
    sendcounts_C[world_rank],
    MPI_LONG_LONG,
    0,
    MPI_COMM_WORLD
);

// 广播完整的矩阵 B
MPI_Bcast(
    B.data(),        // 发送/接收缓冲区
    N * K,           // 元素个数
    MPI_LONG_LONG,   // 数据类型
    0,               // root 进程
    MPI_COMM_WORLD
);
```

偏移量，从矩阵的
哪个位置开始发送

实现将余数3分发给前三个进程

广播完整的B

关键代码分析

```
for (int i = 0; i < local_rows; ++i) {
    for (int j = 0; j < K; ++j) {
        long long sum = 0;
        for (int k = 0; k < N; ++k) {
            // A_local[i][k] * B[k][j]
            sum += A_local[i * N + k] * B[k * K + j];
        }
        // D_local[i][j] = sum + C_local[i][j]
        D_local[i * K + j] = sum + C_local[i * K + j];
    }
}
```

外层：遍历分块后矩阵Ax的行

中层：遍历结果矩阵D的列数

内层：计算核心
A的i行k列元素 × B的k行j列元素



结果性能分析

核心数 (p)	运行时间 (s)	加速比 (S_p)	效率 (E_p)
1	5.400121	1.000	100.0%
2	4.135167	1.306	65.3%
4	3.033115	1.780	44.5%

加速比 (S_p) = T_serial / T_parallel(p)（串行/并行）

效率 (E_p) = S_p / p

- 加速比：4核与2核均快于串行，但加速比有限，并未达到线性加速
- 效率：并行效率随核心数增加而急剧下降，从 2 核的 65.3% 降低到 4 核的 44.5%
- 瓶颈：阿姆达尔定律，总加速比受限于串行部分

分析

```
for (int i = 0; i < local_rows; ++i) {
    for (int j = 0; j < K; ++j) {
        long long sum = 0;
        for (int k = 0; k < N; ++k) {
            // A_local[i][k] * B[k][j]
            sum += A_local[i * N + k] * B[k * K + j];
        }
        // D_local[i][j] = sum + C_local[i][j]
        D_local[i * K + j] = sum + C_local[i * K + j];
    }
}
```

三层循环中，内层遍历的是k，意味着对于矩阵B，采用的是垂直读取，而非内存的行存储读取
跳跃时读取导致Cache命中率极低，花费大量时间把数据送入缓存

方案1：转置矩阵B，把列变为行，强行让内存连续，缺点是需要额外的内存空间

方案2：改变循环顺序，从i-j-k改为了i-k-j，此时最内层循环的是j，对于B[k][j]来说，是按照行读取，顺应cpu的缓存机制

优化1：转置B

```
// 分配转置矩阵空间（注意维度反转：K * N）
// B 是 N * K (2047 * 1023) -> B_T 是 1023 * 2047
vector<long long> B_T(K * N);

// 执行转置
for(int r = 0; r < N; ++r) {
    for(int c = 0; c < K; ++c) {
        B_T[c * N + r] = B[r * K + c];
    }
}

// 计算（使用 i-j-k 顺序，因为转置后这样也是连续访问了）
for (int i = 0; i < local_rows; ++i) {
    for (int j = 0; j < K; ++j) {
        long long sum = 0;
        for (int k = 0; k < N; ++k) {
            // A 按行访问（连续）
            // B_T 按行访问（连续）-> B_T[j][k]
            sum += A_local[i * N + k] * B_T[j * N + k];
        }
        D_local[i * K + j] = sum + C_local[i * K + j];
    }
}
```



优化2：遍历顺序

```
for (int i = 0; i < local_rows; ++i) {
    for (int j = 0; j < K; ++j) {
        long long sum = 0;
        for (int k = 0; k < N; ++k) {
            // A_local[i][k] * B[k][j]
            sum += A_local[i * N + k] * B[k * K + j];
        }
        // D_local[i][j] = sum + C_local[i][j]
        D_local[i * K + j] = sum + C_local[i * K + j];
    }
}
```

计算一个个点，每个点加起来得到sum

```
// // 初始化 D_local 为 C_local 的值，优化1
D_local = C_local;

for (int i = 0; i < local_rows; ++i) {
    for (int k = 0; k < N; ++k) {
        long long r = A_local[i * N + k]; // 提取 A 的值，放到寄存器
        for (int j = 0; j < K; ++j) {
            // 这里的 B[k * K + j] 和 D_local[i * K + j]
            // 随着 j 的增加，都是访问连续内存！
            D_local[i * K + j] += r * B[k * K + j];
        }
    }
}
```

固定用一个A[i][k]，去乘B的第k行，把结果加到D第i行，同时更新1023个变量



结果性能分析

核心数 (p)	运行时间 (s)	优化前	更换顺序	转置B
1	5.400121	10.493875	7.710275	7.662816
2	4.135167	6.659056	3.945504	3.927883
4	3.033115	3.398177	1.224864	1.271494

瓶颈分析

- **瓶颈一：串行 I/O (主要瓶颈)**

  只有0号进程负责读取数据和写入数据，分发数据

  1 核总时间 5.4 秒，4 核仍需 3.03 秒。这说明串行 I/O 占用了大量时间，这部分时间无法通过增加 CPU 来缩短，它为总加速比设下了极低的天花板

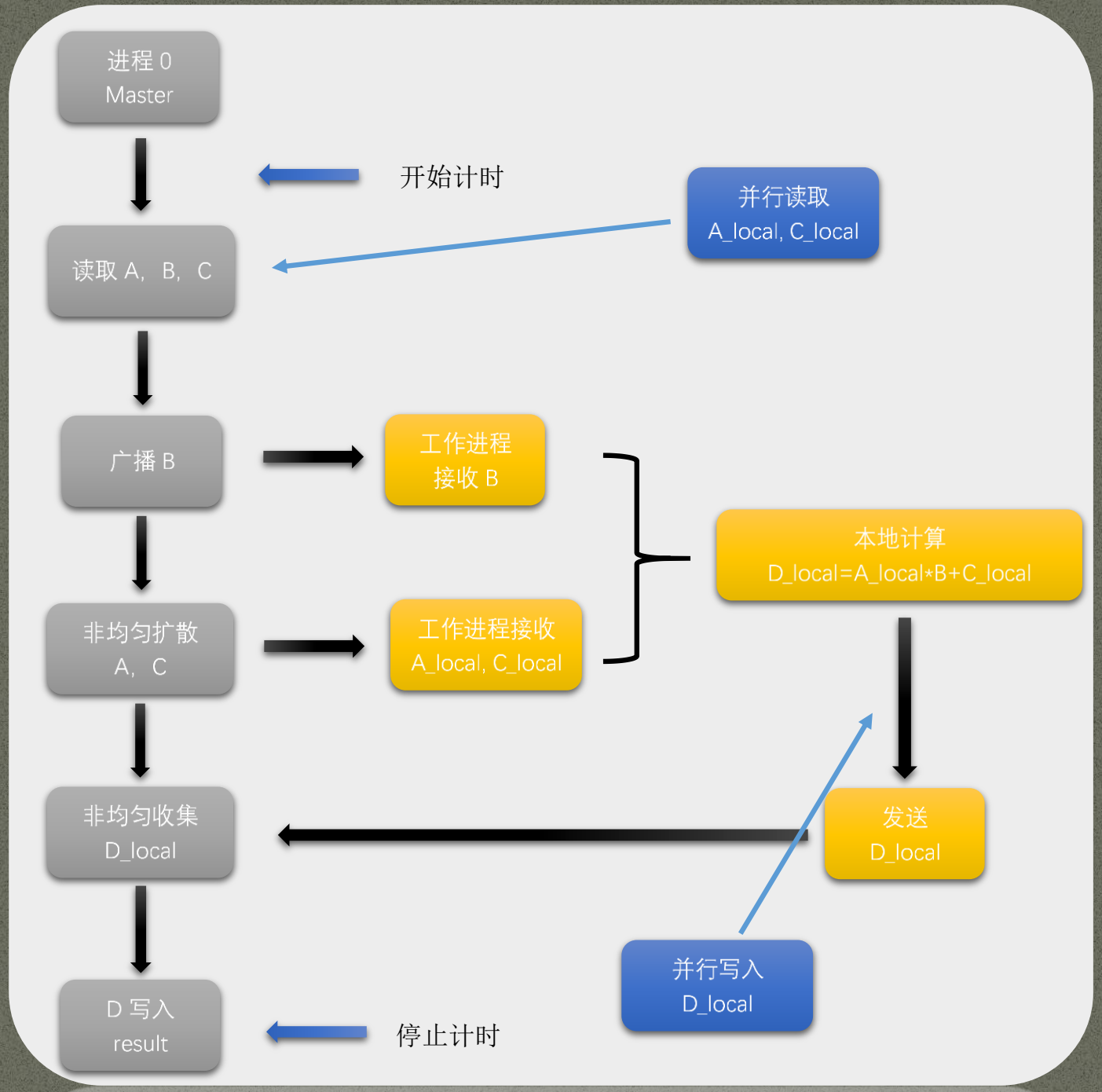
- **瓶颈二：通信开销 (全局广播)**

  必须使用 MPI_Bcast 将完整的 B 矩阵发送给所有进程

  这是一个全局通信操作，所有进程必须等待它完成。随着进程数增加，通信总开销会变大

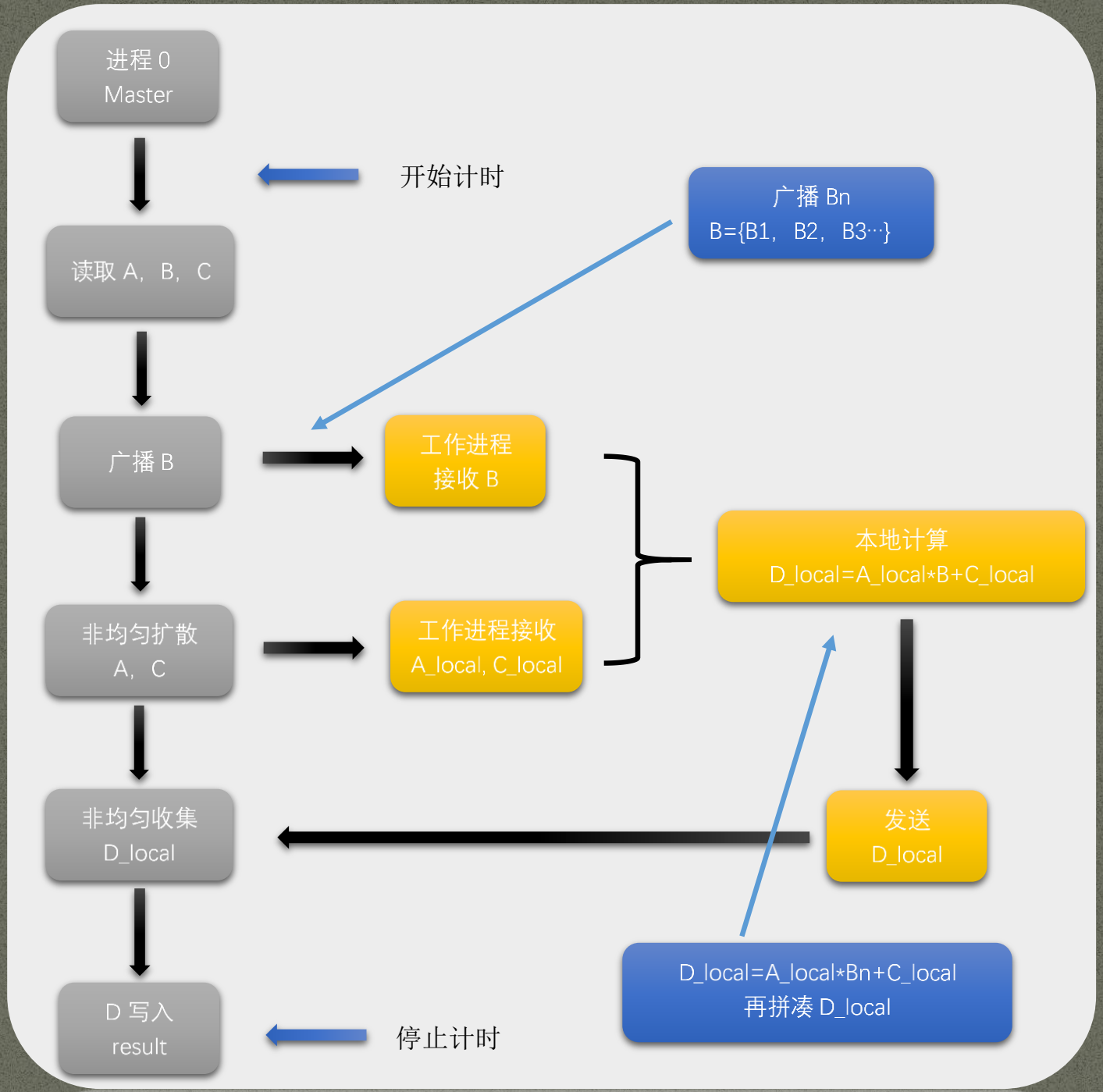
优化：并行I/O

- 核心思想：消除0号进程的串行I/O，让所有进程一起读写文件
- 多进程打开同一文件，每个进程定义自己的视图（块），提高I/O吞吐量
- 潜在问题：非二进制文件，每行字节数不同，无法直接计算偏移量读取，可能需要建立偏移表，记录每行在文件的起始字节，但是需要预先扫描文件，构建表分配行号，类似添加索引，导致新的串行瓶颈



优化：分块广播B

- 核心思想：不再一次性广播整个大矩阵 B，而是将矩阵按行或列，切分成若干小块，逐块广播，每个进程收到部分 B 后即可开始计算对应的局部矩阵乘法
- 使用异步广播，计算当前块的同时传输下一块的数据，最后累加结果，无需等待完整的B传入
- 潜在问题：块划分不能太小，广播次数太多产生额外开销



感谢聆听

2025